Aplicación FIWARE mejorada

Práctica 2 – Gestión de Datos en Entornos Inteligentes

Objetivo

Mejorar la aplicación de la Práctica 1 para **incluir registros y suscripciones NGSIv2, envío de notificaciones desde el servidor a la interfaz**, usando flujo de trabajo GitHub Flow.

Entrega

Un **archivo ZIP con la carpeta de la aplicación** web obtenido **mediante *git archive***. Incluir, además:

- **Conversaciones con el agente (solo al final, para que el agente no las use como contexto)**.
- ***README.md*** con las instrucciones para ejecutar la aplicación y la URL del repo GitHub de la misma.
- ***requirements.txt*** obtenido ejecutando *pip freeze > requirements.txt*

Ajustar *.gitignore* para que no se incluyan ni la carpeta *.venv* ni las carpetas __pycache__ en el archivo ZIP.

Flujo de trabajo GitHub Flow

Cada actuación de implementación se debe llevar hacer usando el flujo de trabajo **GitHub Flow**:

- Con el agente en **modo Plan, elaboramos un plan de implementación** del *issue* que queremos implementar. Cuando lo tengamos finalizado, cambiamos a modo Agente.
- Pedimos al agente que **cree un *issue*** en el repo remoto GitHub con el contenido del plan.
- Pedimos al agente que **cree una rama (*branch*) git** para llevar a cabo la implementación del *issue*.
- Pedimos al agente que **confirme cambios (*commit*) en local y suba (*push*) la nueva rama** al repo remoto.
- Finalmente, pedimos al agente que **cierre el issue fusionando (*merge*) la nueva rama a la rama *main* y que sincronice *(push)*** con *origin/main*. Si no somos los propietarios del repo remoto, le pediremos que cree una PR (*Pull Request*) con la nueva rama. El propietario del repo remoto deberá revisar la PR y fusionarla con *main* para cerrar el *issue*.

Actualizar *PRD.md*, *architecture.md* y *data_model.md* siempre después de finalizar la implementación de un *issue*. Es conveniente indicar esto en *AGENTS.md*.

Para **comenzar a desarrollar nuestra aplicación**:

- Crearemos una **carpeta nueva**, donde **copiaremos los archivos *docker-compose.yml, .env, services, import-data* del tutorial FIWARE NGSIv2 Subscriptions**.
- Con la ayuda del **agente en modo Plan, construiremos el *PRD.md*, *architecture.md* y *data_model.md*** a partir del enunciado de la práctica, para lo cual será conveniente convertir a Markdown el enunciado, eliminando y ampliando los puntos que sean necesarios. Cuanto más detallados, concretos y específicos sean ***PRD.md*, *architecture.md* y *data_model.md***, mejor será la primera versión de la aplicación.
- Pediremos al agente que cree un repo remoto en GitHub para nuestra aplicación y, a partir de los archivos ***PRD.md*, *architecture.md* y *data_model.md***, cree el **primer *issue* consistente en el plan de**

implementación de la primera versión de la aplicación. Tanto para la elaboración de esos tres archivos como del plan de implementación recogido en este primer *issue*, nos ayudaremos del agente en modo Plan, solo cambiando a modo Agente cuando necesitemos que escriba los archivos.

Modelo de datos ampliado

1. Actualizar tipo de entidad ***Employee***. Cada empleado trabajará en un solo *Store*. Añadir los atributos *email*, *dateOfContract*, *skills* ('MachineryDriving', 'WritingReports', 'CustomerRelationships'), *username*, *password*.
2. Añadir al tipo de entidad ***Store*** los atributos *url*, *telephone*, *countryCode* (2 caracteres), *capacity* (metros cúbicos), *description* (texto amplio), *temperature*, *relativeHumidity*.
3. Añadir al tipo de entidad ***Product*** el atributo *color* (tipo *Text* que almacenará el color RGB en hexadecimal).
4. Crear el **diagrama de entidades UML** usando Mermaid y mostrarlo renderizado en el apartado *Home*.

Crear un *script* de **carga de datos inicial** de modo que **haya 4 empleados, 4 tiendas con 4 estanterías cada una, 10 productos y tantos ítems de inventario como sea necesario para que haya al menos 4 productos por estantería**. Podemos indicar al agente que tome el script *import-data* como base. Utilizaremos imágenes disponibles de modo gratuito o generadas mediante un modelo de imagen como Nano Banana 2 o similar.

Proveedores de contexto externo

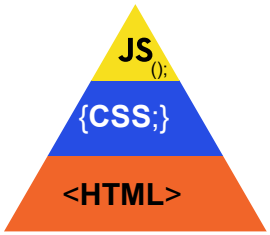
Los atributos ***temperature*** y ***relativeHumidity*** de un *Store* serán proporcionados por un proveedor de contexto externo (la aplicación que corre en el contenedor *tutorial* del tutorial *Context Providers*). El atributo ***tweets*** de un *Store* también será proporcionado por un proveedor externo (de nuevo, la aplicación que corre en el contenedor *tutorial*). El registro en Orion de estos dos proveedores externos debe hacerse al arrancar la aplicación.

Suscripciones a Orion y notificaciones servidor a cliente

En nuestra aplicación recibiremos desde Orion las **notificaciones** contempladas en el tutorial *Subscriptions*, i.e. **cambio de precio de un *Product*** y **bajo *stock* de un *Product* en un *Store***. El alta de estas suscripciones en Orion se efectúa como en dicho en tutorial. Como nuestra aplicación escucha en *localhost*, necesitamos que Orion nos envíe ahí las notificaciones. Orion se ejecuta en un contenedor Docker, con lo que para él *localhost* hace referencia a su propio contenedor. Para hacer referencia al *localhost* de la máquina anfitriona desde Orion debemos usar *host.docker.internal* en lugar de *localhost*.

Enviaremos **notificaciones desde el servidor al navegador para que se actualicen los elementos de la interfaz** de usuario involucrados. Usar Flask-SocketIO en el servidor y Socket.IO en el navegador.

Interfaz de usuario HTML + CSS + JS



Principios generales:

- Cuando algo se pueda hacer tanto mediante CSS como mediante JS, usar CSS.
- Evitar al máximo generar código HTML en el código JS. Siempre que se pueda, el código JS deberá actualizar el valor de los atributos elementos HTML ya presentes en la página en lugar de añadir nuevos elementos.

Para implementar los aspectos visuales que se piden, consultar la **sección HowTo de W³ Schools**, que contiene multitud de **"recetas"** a este respecto.

La interfaz de la aplicación debe soportar **inglés y castellano**. También debe incluir un *toggle* para cambiar entre **modo Dark y Light**.

Formularios de entrada de datos

1. Utilizar el mayor número de elementos HTML `<input>` distintos.
2. Incluir las **reglas de validación HTML y JS** apropiadas.

Estructura de la interfaz

1. Vistas ***Products***, ***Stores*** y ***Employees***
    a. Mostrar la lista de entidades como una **tabla** donde **cada entidad incluirá su imagen, su nombre y dos enlaces tipo botón de borrado y modificación**. Además, cada entidad en la tabla mostrará los siguientes **atributos específicos**:
    - *Product*: *color*, *size*
    - *Store*: *countryCode*, *temperature*, *relativeHumidity*
    - *Employee*: *category*, *skills*

    b. Añadir al principio de la vista un enlace tipo **botón para añadir una nueva entidad**.
2. Vista ***Product***
    a. Mostrar la **tabla de *InventoryItems* de ese *Product* agrupada por *Store***: para cada *Store*, mostrar una fila con el nombre del *Store* y el valor de *stockCount* para ese producto y, a continuación, las filas de con los valores de *shelfCount* para las distintas *Shelfs* que contienen ese *Product*.
    b. En cada encabezado de grupo *Store* añadir un botón tipo enlace que permita **añadir un *InventoryItem* con ese *Product* a otra *Shelf*** que no sea ninguna de las que ya contiene ese *Product*. Los posibles valores de la *Shelf* para el nuevo *InventoryItem* deben restringirse mediante un elemento tipo *select* que cargue dinámicamente las *Shelfs* de ese *Store* que todavía no contengan ese *Product*.
3. Vista ***Store***
    a. Mostrar la ubicación de la tienda en un **mapa mediante Leaflet JS**.
    b. Incluir un **recorrido inmersivo virtual por tienda** mostrando sus estanterías con los productos que tienen almacenados, indicando para cada producto el número de unidades en esa estantería y el total de unidades en stock. Usar **Three JS**.
    c. Mostrar la **tabla de *InventoryItems* agrupada por *Shelf***: para cada *Shelf*, mostrar una fila con su nombre y su nivel de llenado (utilizar una barra de progreso) y, a continuación, las filas con los *Products*, incluyendo sus atributos *image, name*, *price*, *size*, *color*, *stockCount* y *shelfCount*.
    d. Añadir un botón tipo enlace que permita **añadir una *Shelf* a ese *Store***.
    e. En cada encabezado de grupo *Shelf* añadir un botón tipo enlace que permita **modificar esa *Shelf***.
    f. En cada encabezado de grupo *Shelf* añadir un botón tipo enlace que permita **añadir un *InventoryItem* de otro *Product*** que todavía no esté presente en esa *Shelf*. Los posibles valores de *Product* para el nuevo *InventoryItem* deben restringirse mediante un elemento tipo *select* que cargue dinámicamente los *Products* existentes que todavía no estén presentes en esa *Shelf*.

g. Incluir en cada *InventoryItem* un botón tipo enlace que permita **comprar una unidad de ese producto**. Para ello, utilizar la siguiente petición a Orion:

PATCH */v2/entities/<inventoryitem_id>/attrs*

Body:

```
{
      "shelfCount": {"type":"Integer", "value": {"$inc": -1}},
      "stockCount": {"type":"Integer", "value": {"$inc": -1}}
}
```

h. Incluir la información de la **temperatura y humedad** relativa utilizando iconos y colores diferentes en función de los valores que tomen. Mostrar el valor numérico al lado del cada icono.

i. Incluir los ***tweets* de ese Store** después de la tabla de *InventoryItems*. Usar un icono que recuerde a X (Twitter) a la izquierda de cada *tweet*.

j. Incluir un **apartado de notificaciones**, donde se mostrará cada notificación que se reciba (p. ej. la notificación de *stock* bajo para alguno de los productos del *Store*).

4. Cuando se reciba una **notificación de cambio de precio** de un producto, **reflejar dicho cambio en todas las vistas** donde aparece.

Aspectos visuales

1. Vista ***Employee***: incluir la **foto del empleado** con una transición CSS que **amplie la foto** cuando se pasa el ratón por encima de ella.
2. Vista ***Store***: incluir una **foto del almacén** con una transición CSS que **amplíe la foto** y, al mismo tiempo, una animación CSS que **rote la imagen 360 grados**.
3. Barra de progreso del nivel de llenado de una ***Shelf***: utilizar distintos **colores según el nivel de llenado**.
4. Utilizar **colores, iconos, elementos visuales, etc. para compactar la información mostrada**, especialmente en las vistas con **tablas**. Utilizar, p. ej. los iconos de Font Awesome, escogiendo de modo adecuado. P. ej. los siguientes atributos se pueden mostrar de esta forma:
   - *color*: con un cuadrado del color correspondiente.
   - *country*: con un icono con la bandera del país.
   - *category*: con un icono distinto según la categoría.
   - *skills*: con iconos que muestren los distintos valores.
5. **Barra de navegación**: deberá resaltar la sección (*Home / Products / Stores / Employees*) activa en cada momento y permanecer visible cuando se efectúe *scroll*.
6. Añadir **pestaña *Stores Map*** a la barra de navegación. Esta vista mostrará las imágenes de los *Stores* sobre el mapa. Usar la librería Leaflet JS. Cuando se pase el ratón sobre un Store, se deberá mostrar una tarjeta (imagen + texto) con sus atributos principales. Cuando se pulse sobre un *Store*, se deberá acceder a su página de detalle.